Claude Code Master Build Prompt

VERSION 1.1 | 06 NOVEMBER 2026

400 COFFEE

Claude Code Master Build Prompt

VERSION 1.1 | 06 NOVEMBER 2026 | CUSTOMER EXPERIENCE UPDATE

Paste this complete instruction into Claude Code after opening the actual repository.

ROLE AND DELIVERY STANDARD

You are Claude Code acting as a senior product engineering team: principal frontend engineer, 3D web experience engineer, backend integration engineer, accessibility specialist, QA lead and release engineer. Build the 400 Coffee customer-facing experience described here. The approved direction is interactive + 3D manager + cinematic camera. Do not reinterpret it as a conventional dashboard, generic e-commerce site, plain QR menu or static mockup.

SCOPE BOUNDARY

Build customer side only. Do not redesign or implement cashier POS, kitchen display, owner dashboard or platform admin in this task. You may implement typed API contracts, mocks and integration adapters needed by the customer side. Do not claim backend, payment, WhatsApp or restaurant synchronization is live unless credentials, endpoints and successful end-to-end tests prove it.

MANDATORY WORKING METHOD — NO ONE-SHOT CODE DUMP

1. Start by inspecting the complete repository: file tree, package manifests, framework, current routes, design system, backend contracts, environment variables, existing tests, lint/typecheck/build scripts and deployment assumptions. Do not overwrite existing architecture without a reason.

2. Produce a concise implementation plan, dependency map, file-change plan and risk register. Identify unknowns explicitly. Never invent API endpoints, credentials, provider capabilities or menu content.

3. Create a working baseline commit/branch or a recoverable checkpoint before major changes. Do not delete working code to make a clean rewrite.

4. Implement in small vertical slices. After each slice, run typecheck, lint, unit tests and the production build. Fix all regressions before proceeding.

5. Use real browser testing (Playwright or repository's existing browser test framework). Test at mobile 360×800, 390×844; tablet 768×1024; desktop 1440×900. Capture screenshots for each important state.

6. Continue the build → run → inspect → fix → retest loop until every applicable acceptance criterion passes. Do not stop after the first visually plausible screen.

7. If external integration credentials are absent, build a clearly labelled sandbox/mock adapter and contract tests. Record the integration as BLOCKED / NOT CONNECTED; never simulate real success in production mode.

8. Before declaring done, provide a file-change summary, commands run, test results, screenshots, known limitations, configuration required and exact local run instructions. Do not claim a test passed unless it was executed.

PRODUCT EXPERIENCE — FULL CUSTOMER JOURNEY

A. Entry and cinematic café arrival

Customer scans outlet/customer QR or opens the PWA link. Do not require app installation or login.

Start with a cinematic camera push/dolly into a warm, premium café: dark walnut counter, espresso machine, café shelves, brass details, amber practical lights, subtle steam/dust motes and depth.

A 3D café manager stands behind the counter. The manager welcomes the visitor with the exact default copy: “Welcome to 400 Coffee. How can I help you today?”

Show two unmistakable primary choices: “View menu & place order” and “Pay bill”.

Camera settles before customer interaction. Add visible “Skip intro” and accessible focus order. Never trap user in an animation.

Use subtle idle motion (breathing, blink, natural hand gesture). Avoid uncanny exaggerated lip sync. Dialogue is scripted/state-driven, not fake AI.

Provide sound only if explicitly enabled by the user; never autoplay audio.

If WebGL/3D fails or reduced motion is preferred, show an elegant 2D café scene with same actions and same content. No feature may depend on GPU.

B. Diary menu

“View menu & place order” transitions into a leather-bound vintage café diary. Use tactile page-turn or tasteful crossfade, with responsive layout.

Categories are configurable; initial suggested categories: Coffee, Hot Beverages, Cold Beverages, Fresh Juices, Sandwiches, Snacks, Desserts.

Each product displays approved photo, name, description, price, availability and relevant dietary/allergen metadata only if verified.

Menu content, price, tax and availability are fetched from the backend. Any seed/sample menu must be visibly demo-only. Never hardcode production price or tax.

Keep basket count visible. Support category switching, optional search, unavailable state and loading skeleton.

C. Product customizer

On item selection, open a premium product detail/customization panel.

Support product variants/sizes, allowed modifiers/add-ons, quantity stepper, preparation/cooking instructions (e.g. less spicy, extra toasted, no ice), and optional special request text.

Show each price delta and the updated line total. Prevent unsupported modifier combinations and enforce server constraints.

Do not assume all products have sizes or add-ons. The schema drives the UI.

D. Basket

Show each item, quantity, size, modifiers, notes, unit price and line total.

Allow edit, remove, quantity changes and return to menu.

Show item subtotal, configured tax/fees/discounts, payable total and fulfilment details. Use server quote as source of truth.

Preserve basket state across navigation and recoverable refresh. If catalog changes make an item unavailable or price changes, explain and require customer review.

E. Customer details

Guest checkout by default; no forced account creation.

Ask customer name and mobile number, with clear reason: order identification and requested transactional updates.

Validate Indian mobile number on client and server. Do not log raw phone numbers.

Obtain explicit transactional WhatsApp consent where required; keep optional marketing consent separate and unchecked by default.

F. Final review

Show complete basket, modifiers, instructions, contact details, fulfilment mode and full price breakdown.

Manager asks: “Does everything look right?” and “Would you like to pay now, or at the takeaway counter?”

Provide “Pay now” and “Pay at takeaway counter” choices. If dine-in is enabled, display its configured fulfilment options; do not invent table numbers.

Edit/back must preserve all state.

G. Pay later

Create the order with a payment-due/unpaid state.

Display order reference, submitted/awaiting cashier acceptance state and “Payment is due at the takeaway counter.”

Never mark this order paid. Restaurant order appears in backend workflow based on configured cashier acceptance/release policy.

H. Pay now

Display only configured methods: UPI, QR, credit card, debit card.

Start payment intent/session on server. Use provider-hosted checkout, deep link, dynamic QR or terminal flow as supported.

Never collect/store PAN or CVV. Never trust browser return, screenshots or user declaration as payment truth.

Only show PAID after verified server-side webhook/status API or authorized manual merchant verification. Match expected amount/currency/order reference. Handle pending, failed, expired, retry and reconciliation states.

A static UPI QR only identifies merchant; it is not an order-specific verified payment. If only static QR exists, show instructions for manual verification and retain pending verification until staff verifies in merchant-side channel.

I. Confirmation and updates

Show unique public order reference, exact item summary, amount, fulfilment instructions, order status and payment status.

Distinguish submitted/awaiting acceptance, accepted, preparing, ready and completed.

Queue WhatsApp transactional confirmation only after durable order creation and appropriate consent. Payment confirmation only after payment verification.

Show message delivery state accurately. If WhatsApp provider is unavailable, the order remains valid and the UI must not falsely say message sent.

Customer order tracking must recover after refresh and reconnect.

J. Pay bill path

Bill lookup asks for bill/order reference and phone verification factor.

Server returns outstanding balance. Never let the customer type an arbitrary amount in production. Client cannot change server outstanding amount.

Pay using configured provider; verify server-side. Handle already-paid, partial, expired and not-found states without exposing another customer's data.

VISUAL SYSTEM — DO NOT DEVIATE

Brand mood: premium cinematic café, warm, tactile, intimate, editorial.

Core palette: espresso-black #120D0A, walnut #241810, roasted brown #57331F, brass/amber #D9AD70, parchment #F1E4CC, muted warm gray #B8A48B.

Materials: aged leather, dark walnut, brushed brass, ceramic, glass, paper. Warm practical lighting, soft volumetric glow, subtle steam.

Type: editorial serif for hero/menu titles; high-legibility sans serif for controls, prices, labels and statuses. Use supplied/licensed fonts only.

Do not use generic SaaS dashboard cards, neon cyberpunk, excessive glassmorphism, emoji as final product art, random gradients, loud animations or tiny controls.

Product imagery must be consistent, high quality and licensed/merchant supplied. No broken remote hotlinks.

3D manager and café environment must be coherent and polished. Prefer optimized glTF assets and baked lighting; maintain a low-cost fallback.

The six supplied UI concept screenshots are direction references only, not production assets and not proof of implementation. Preserve the core composition, palette and information hierarchy while making responsive and accessible adaptations.

Make every state polished: loading, empty basket, unavailable product, validation error, network offline, payment pending/failure, order awaiting acceptance and successful confirmation.

TECHNICAL REQUIREMENTS

Respect repository stack. If no stack exists, recommend and document React + TypeScript PWA; Three.js / React Three Fiber for 3D only after mobile performance feasibility is proven.

Separate scene/presentation state from order/payment domain state. Scene animation must never own order truth.

Build reusable modules: customer shell/session, scene, manager dialogue, diary menu, catalog client, customizer, basket, identity/consent, review, checkout/payment adapter, bill lookup, order confirmation/status, telemetry.

Typed API client, schema validation, error boundaries, cancellation/timeout handling, retry/backoff for safe reads, idempotency keys for order/payment writes.

Server owns menu price, tax, availability, order totals, order lifecycle and payment state.

Tenant/outlet scope must come from a signed/validated QR/session context; never trust arbitrary tenant_id from browser.

Protect public APIs with rate limits, anti-enumeration, validation, CSRF/CORS/session protections and secure headers.

PII must be excluded from analytics/error logs. Use opaque public refs, not sequential internal IDs.

Respect prefers-reduced-motion, keyboard access, focus management, labels, contrast and screen-reader status announcements.

Performance: lazy-load scene and 3D assets, use compressed textures/models, optimize draw calls, pause animation when tab hidden, adapt quality on low-end devices. Core menu/checkout must remain usable while 3D assets load.

Provide feature flags for immersive scene, payment methods, WhatsApp and customer tracking.

Provide .env.example with names and explanations only; never commit secrets.

SUGGESTED API CONTRACTS (DO NOT ASSUME THEY EXIST)

GET /public/outlets/{slug}/experience-config

GET /public/outlets/{slug}/menu

POST /customer/sessions

POST /customer/orders/quote

POST /customer/orders (idempotency key)

GET /customer/orders/{publicRef}

POST /customer/orders/{publicRef}/payment-intents

GET /customer/payment-attempts/{id}

POST /customer/bills/lookup

POST /customer/bills/{publicRef}/payment-intents

Treat these as proposed contracts. Inspect backend first. If absent, write OpenAPI/spec and mock adapter, then clearly document integration work.

STATE AND PAYMENT INVARIANTS

Order and payment are separate state machines.

Order submitted ≠ cashier accepted.

Order accepted ≠ payment received.

Payment intent created ≠ paid.

Browser redirect success ≠ paid.

Static UPI QR scan ≠ paid.

WhatsApp message sent ≠ order accepted.

Every mutation must be idempotent where retries are possible.

Every critical state transition is server-authoritative and audited.

A parent order may include separate kitchen station tickets; the customer does not need to know station internals but must see one consistent order status.

REQUIRED TESTS

Journey from entry to pay-later confirmation.

Journey from entry to verified payment in provider sandbox.

Bill lookup/payment with valid and invalid reference/phone.

Menu API unavailable, slow and partial failure.

Product becomes unavailable after adding to basket.

Price changes before submit; server quote updates and customer reconfirms.

Invalid mobile and consent handling.

Duplicate taps and retry/network loss around order submission.

WebGL failure, low GPU, reduced motion, mobile portrait and landscape.

Payment pending, webhook delayed, duplicate webhook, invalid signature, amount mismatch, expired, failed and refund states.

WhatsApp provider outage and retry.

Restaurant sync delay, duplicate event and reconnect recovery.

Accessibility keyboard-only and screen-reader flow.

No horizontal scroll and no clipped CTA at all target breakpoints.

Security: tenant/outlet isolation, bill lookup enumeration, XSS/input injection, rate limit and PII logging.

Visual regression screenshots at each journey milestone.

FINAL RELEASE GATE

Do not declare production-ready until all P0/P1 tests pass, real payment provider and WhatsApp integration status is explicitly verified, no fake payment success is possible, 3D fallback works, order sync has recovery, legal copy and menu content are approved, and client/merchant sign-off is recorded. If any dependency is missing, label the release “prototype / integration pending” and list the exact blocker.